AVERT (Authorship Verification and Evaluation through Responsive Testing): an LLM-Based Procedure that Interactively Verifies Code Authorship and Evaluates Student Understanding

Florentin Vintila
University of Applied Sciences
Stuttgart, Germany
florentin.vintila@hft-stuttgart.de

2024 21st International Conference on Information Technology Based Higher Education and Training (ITHET) | 979-8-3315-1663-5/24/$31.00 ©2024 IEEE | DOI: 10.1109/ITHET61869.2024.10837675

***Abstract*— The rapid development of generative artificial intelligence challenges traditional plagiarism detection systems. Classic methods, based primarily on similarity, are more and more ineffective against content that is AI-generated, and which resembles authentic student submissions. This requires the use of detection methods that not only establish authorship but also augment student engagement and motivation, thus discouraging future plagiarism attempts. In this work-in-progress paper, we present AVERT (Authorship Verification and Evaluation through Responsive Testing), a proof-of-concept approach for establishing the authorship of programming assignments. Integrated in an LLM-based chatbot, it proactively assesses students' understanding of programming assignments by interactively questioning them about the process of developing their own code. This way the reasons for which students may be dishonest, such as motivation and lack of subject comprehension, are addressed. Using advanced prompting techniques and formulating questions that employ the Socratic method, AVERT dynamically generates questions to verify the students' understanding of the coding process. By asking students to explain the logic behind their solutions, it also improves analytical skills and engages students in the learning process. By automating the assessment of authorship, AVERT alleviates the logistical problems encountered by educators in large classroom environments where individualized assessment demands large resources. It also helps instructors in grading students by making the details of each conversation available for evaluation. AVERT provides a scalable and efficient method for establishing authorship, overcoming the shortcomings of other techniques by directly addressing the primary goal of programming assignments: assessing students' abilities to write functional code.**

Keywords*— *Academic Integrity, Large Language Models, Question Generation, Socratic Method, Computer Science Education, Plagiarism Detection

I. INTRODUCTION

The integration of technology into academia has brought with it new challenges to academic integrity. The internet's development, and its concomitant expansion of easy-to-access academic resources, has made information more accessible. Academic Institutions have always been engaged in efforts towards nurturing environments based on adherence to academic honesty policies. Still, as reported by various researchers, concerns about preserving the integrity of learning have grown along with an escalation in the number of students cheating. The unwillingness of students to confess to any dishonesty adds more uncertainty in determining the spread of plagiarism [1]. Plagiarism frequency estimates differ widely, with some research showing rates as high as 80% and calling for efficient mechanisms of deterrence [2], [3]. Furthermore, there was a significant increase in the number of scientific articles dealing with all types of academic misconduct from 2000 to 2015, with a subsequent exponential rise through to the present day reflecting the significance with which higher education had responded to these challenges during that period [4]. This growth indicates increased awareness in academia surrounding misconduct and highlights how difficult it has been dealing with this issue, at first due to the influence of the expansion of online platforms, and later, advances in Artificial Intelligence (AI).

The tipping point came in recent years with the advent of generative artificial intelligence based on Large Language Models (LLMs), like ChatGPT, which can generate written content that responds to queries at a level nearly indistinguishable from that of a human writer [5]. On one hand, such technology provides personalized assistance to students, making them more motivated and engaged [6]. On the other hand, the ease with which AI can generate content might lead to excessive reliance on technology, leading to less developed critical thinking and problem-solving abilities among students [7]. This could promote academic dishonesty, including new forms of plagiarism and fraudulent activities that involve misrepresenting authorship [8].

LLMs must meet certain requirements when processing source code, compared with natural language. They must follow exact syntax specifications to generate functional code. The source code has a different structure and logic, unlike natural language, and this is reflected in how they are processed. Traditional plagiarism detection approaches that use similarity between two existing sources find it difficult to identify code produced by LLMs, which can generate original and diverse content. This situation causes concern among educators regarding the authorship of the code, leading them to adjust their methods of evaluating student work to ensure academic integrity. Sole reliance on technical solutions to combat cheating is not a sustainable strategy, as it may only perpetuate a cycle of increasing evasion techniques [9]. Therefore, academic institutions need to implement educational strategies that address the root causes of academic dishonesty: time pressure and fear of failure [10]. More than just preventing plagiarism, these strategies substantially boost student involvement in the learning process.

Seeking to bridge the gap in proactive plagiarism prevention, we introduce in this work-in-progress paper the conceptual prototype of AVERT (Authorship Verification and Evaluation through Responsive Testing), an approach that is part of an AI chatbot designed to assist students with programming assignments. AVERT generates questions that assess students' understanding of the code development process. Using advanced prompting techniques and formulating questions employing the Socratic method, AVERT dynamically generates questions to verify the students' understanding of the coding process. The entire dialogue is recorded and made available to instructors as part of the assignment assessment. To the best of our knowledge, AVERT is the first approach to establish authorship of source code by verifying the understanding of the development process. It also redefines the role of plagiarism detection in the

I wish to express my gratitude to the KNIGHT project for the generous support and funding which made this research possible. The KNIGHT project is funded by the German Federal Ministry of Education and Research under grant number 16DHBKI072.

programming education by creating an environment that discourages plagiarism.

A. Research Goal

The primary objective of this work-in-progress research is to evaluate the feasibility of a system based on a Large Language Model (LLM) for verifying the authorship of programming assignments. This system employs Socratic questioning to determine if students truly understand the development process of their own code. Utilizing advanced prompting techniques, the LLM generates question templates that are adapted to the Socratic method. Finally, the LLM assesses the students' answers using a flexible scoring system, which is tailored by instructors to meet the specific requirements of the assignment.

B. Research Objectives

To fulfil the main objective of this research, we pursue the following objectives:

1) Develop an approach using Large Language Models (LLMs), such as AVERT, that can reliably determine the authorship of source code by analysing the understanding of the code development process.

2) Identify and address anticipated challenges in the development of AVERT.

3) Evaluate the extent to which AVERT can prevent academic dishonesty.

The rest of the paper is organized as follows: Section II describes related work and some existing approaches. Section III details the conceptual framework, which includes question generation, prompt design, and assessment methods. Section IV describes how we will conduct the evaluation of the AVERT system. Finally, Section V concludes the paper by summarizing the findings and discussing future directions for this research.

II. RELATED WORK

A. Plagiarism Detection

One of the greatest challenges in academia is plagiarism that affects the learning process with immediate and long-term effects on students. It hinders the development of critical thinking and analytical skills, with a strong correlation being observed between academic failure and plagiarism. It erodes trust and confidence between students and faculty members. Left undetected, it can cause significant harm to the academic reputation of the institutions concerned undermining its integrity and the value of its education standards [11]. Source code plagiarism detection differs substantially from natural language plagiarism detection in terms of content and structure requirements. While classical source code plagiarism detection relies heavily on structural and syntactical analysis, natural language plagiarism involves assessment of semantic and contextual similarity, which requires understanding the meaning and intention behind the text [12]. There are different types of code similarity detection techniques based on their approach to analysing structural and flow aspects of code. These categories include text-based, token-based, tree-based, program dependence graph (PDG)-based, metric-based, and hybrid methods [13], [14], [15]. Various automatic detection tools have been produced using similarity checking techniques, the most cited in the literature include Moss, JPlag, SIM, Plaggie, and Sherlock [16]. The students tried to evade originality checkers through different lexical and structural changes, thus creating significant hurdles for detection tools. They have limited capability to detect deeper structural forms of plagiarism, especially if the plagiarized code has been significantly altered syntactically but retains similar functionality [17], [18]. Additionally, the methods that use abstract syntax trees and program dependence graphs are computationally intensive and necessitates large resources, which limits their usage in large-scale applications. Hybrid methods that address these problems, combining multiple detection strategies, are cumbersome, requiring significant fine-tuning to balance effectiveness and efficiency [19].

Source code obfuscation is experienced differently between beginners and experts: whereas beginners would often use stylistic modifications such as renaming the variables and syntactic rearrangements, the latter category could implement semantic changes such as converting an iterative process into a recursive one or translating the program into another programming language [20]. Traditional plagiarism detection tools primarily rely on similarity checking to detect cheating. This approach was effective before the widespread use of the internet, when most cheating involved copying directly from peers. The growth of the internet has seen more students accessing external resources for help, thus leading to codes that do not resemble what their classmates have presented [21]. Consequently, various new techniques have been developed that utilize various metrics to detect a broader range of cheating behaviour. These methods provide stronger evidence of cheating and reduce false accusations that arise from similarity checks. Detection tools now incorporate metrics like points rate, style anomalies, inconsistencies, IP address anomalies, and code alterations to improve the accuracy of identifying academic dishonesty [22]. Moreover, some strategies are focused on tracking the coding process itself. This is done by keeping an eye on how the code changes with time such as comparing two commits of a student or monitoring live code changes [23]. The advent of Large Language Models like ChatGPT has brought about significant challenges in academia by allowing students to complete assignments with minimal personal effort. These models provide ready-made solutions, which might reduce the necessity for students to actively engage with and think through their assignments. As a result, there has been an increase in academic dishonesty where most students submit AI generated work instead of their own thereby skipping the learning process. While many tools for detecting AI-generated content focus on natural language, there is relatively little attention paid to AI-generated code. This difference mainly emanates from progress made in AI technologies that facilitate code generation which highlights the need for new instruments designed specifically for coping with specific peculiarities typical of AI-generated codes [24]. Detection of AI-generated code has different requirements compared to other kinds of AI generated texts because it requires functional correctness and must consider the variety of programming languages and code obfuscation techniques. Unlike regular text, where stylistic and thematic consistency can provide clues, code must execute correctly and efficiently [25]. However, as previously discussed, these systems are still far from offering full protection against all tricks that students might apply to avoid plagiarism detection by the available detection tools, even though considerable progress has been made in developing AI-based applications for code plagiarism

detection. It is anticipated that student strategies to circumvent detection tools will change with the rapid development of generative AI systems [26]. Consequently, authorship verification should shift from simply detecting plagiarism to proactively preventing it. A proactive approach implies creating an informed, inclusive, connected, and supportive learning environment that considers students' learning needs, thus naturally discourages academic dishonesty [27], [28].

This paper advocates for a proactive detection method that shifts the focus from merely analysing the source code to understanding the process of its development. This approach can both preserve academic integrity and enhance learning outcomes. AVERT is designed to actively encourage students to individually develop programming skills from the very first assignment, helping to establish independent coding practices early on that are necessary for long-term success.

III. CONCEPTUAL FRAMEWORK

AVERT is integrated into a proactive LLM-based chatbot tutoring system designed to assist students with programming assignments. This system utilizes a multi-agent framework using Langchain technology, which allows developers to create agents capable of reasoning through problems and decomposing them into manageable sub-tasks [29]. Langchain gives the possibility to access students' previous interactions with and behaviour, adjusting the feedback on the student's personal profile. As a specialized agent AVERT evaluates how well students understand the process of developing code to verify if they wrote the code themselves. Initially, each student's code submission is automatically tested for functionality. Submissions that pass these tests proceed to AVERT's authorship verification process, ensuring that only students who have successfully completed the assignment are evaluated for authorship. Students whose submissions fail are supported by other agents within the system, that offer hints, explanations, and guidance to help resolve the assignments. During the phase of authorship verification the chatbot asks questions based on the submitted code and the students' responses. If there is a high probability that the students did not write the code themselves, the chatbot suggests rewriting the code with assistance from other specialized agents. The entire dialogue is recorded and provided to instructors as part of the assignment assessment.

A. Question Generation

Authorship verification using LLMs has other demands compared to code summarization or code comprehension. Whereas code summarization, offers a general overview without going into details generating brief, user-friendly comments that capture the essence of code segments [30], [31], authorship verification adds new requirements that go beyond understanding the code's functionality revealing through questions how the code was conceived, developed, and refined by the student. This approach helps differentiate students who only understand how the code works from those who created it. The central functionality of AVERT lies in its ability to understand source code and generate questions for establishing the authorship of the code. While previous research has concentrated on generating reactive questions from given inputs, AVERT uses a strategy based on the Socratic method that starts with general questions that assess basic understanding and progressively moves to more detailed questions based on the student's responses.

We employ a proactive approach based on the Socratic method, a form of logical inquiry that aims to stimulate critical thinking and encourage students to explore and articulate their reasoning. It serves as a framework for encouraging deeper reflection and understanding by cultivating reasoning skills and guiding students toward insight to assess their comprehension. Socratic questioning encourages reflection, placing greater emphasis on the process of exploring understanding of the topic in discussion rather than merely arriving at a conclusion [32].

The questioning process is iterative and unfolds as follows:

1) The instructor poses a question to the students about a specific aspect of the code development process.

2) The students respond based on their experience and understanding of code development.

3) The instructor follows up with further questions, challenging the students to justify and elaborate on their reasoning.

In the context of verifying students' understanding of the source code development process, the Socratic method is particularly effective because it moves beyond superficial understanding and requires students to explain the logic behind their code, their decision-making process, and the assumptions they made during development. This approach helps assess not only the correctness of the code but also the depth of the student's comprehension and their ability to engage in problem-solving. Experience has shown that it can be challenging for instructors to engage students in online learning. In this respect, the Socratic method has the advantage of keeping students actively engaged in conversation.

The six types of Socratic questions are [32]:

- **Conceptual Clarification Questions** probe the concepts behind arguments, prompting for more explanation and details about previously made statements.
- **Questions Probing Assumptions** question the underlying assumptions that support an argument.
- **Reasons and Evidence Questions** provide a rationale for arguments and require examples that support specific claims.
- **Perspective and Viewpoint Questions** facilitate exploring issues from multiple angles and take various viewpoints into consideration.
- **Implications and Consequences** Questions consider what might happen as a result of certain assumptions or actions.
- **Reflective Questions** turn the question in on itself to probe for a deeper meaning behind the question.

The steps for determining authorship using the Socratic method are as follows and are illustrated in the flow diagram shown in Fig. 1.

- **Student Submission:** Input node where the student submits functional code for verification.

- **Socratic Method Questioning Process:** A series of dynamic question types, progressing from general to specific, adapting based on student responses. These questions include Conceptual Clarification, Probing Assumptions, and Reasons and Evidence. Responses from the student guide the flow of the questions, with simpler questions used if the student encounters difficulty and more complex ones if the student demonstrates deeper understanding.
- **Dynamic Adjustment of Questions:** Based on the student's responses, AVERT adjusts its questioning strategy. For example, it may shift to Perspective and Viewpoint Questions to explore different approaches or to Implications and Consequences Questions, prompting students to consider alternative approaches and the effects of their coding decisions.
- **Reflective Assessment:** The final step in student interaction involves Reflective Questions, encouraging students to review any challenges faced during coding and how they resolved them.
- **Evaluation Process Conclusion:** If the assessment indicates a high probability that the student is not the original author, they are referred to another agent for assistance in rewriting the code. Otherwise, if the assessment confirms the student's authorship, they are provided with personalized recommendations for improvement. All interactions are recorded and logged.
- **Instructor Review:** The entire dialogue is recorded and made available for instructor review, providing detailed insights into the student's understanding and code authorship.

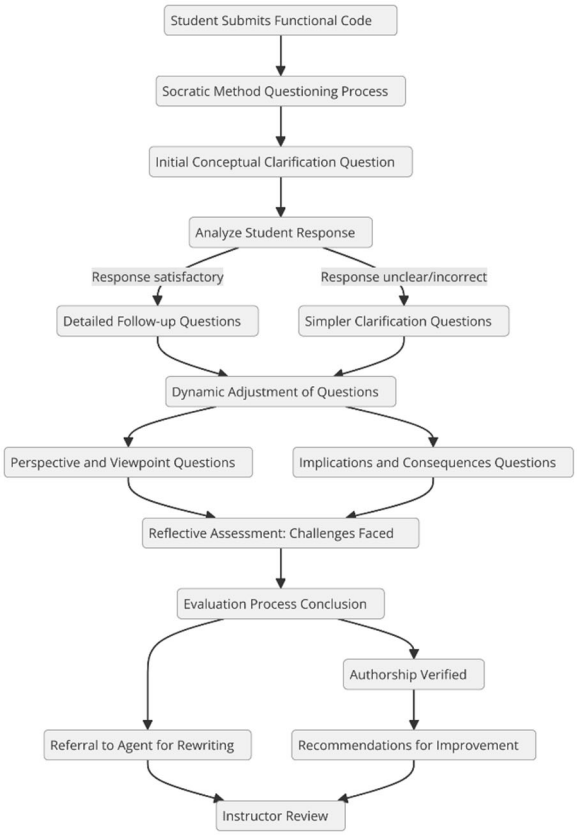

Fig. 1. AVERT Socratic Evaluation Flow.

B. Prompt Design

In practical applications, the prompt serves as the input to the model, and the way it is engineered can significantly influence the resulting output. Adjustments to the structure and content of the prompt can markedly affect the responses generated by the model [33]. To elicit the desired response from a large language model (LLM) while minimizing issues such as incorrect outputs, hallucinations, and non-deterministic behaviours, it is essential to employ effective prompt engineering techniques. Reference [34] provides a comprehensive survey of prompt engineering, exploring various approaches and methodologies from different perspectives. LLM performance significantly improves for simple tasks. Therefore, we first decompose the assignment first in simple elementary tasks using prompt chains. In a chain, a problem is broken down into several smaller sub-tasks, each mapped to a distinct step with a corresponding natural language prompt [35]. For each sub-task, we use few-shot prompting to provide the LLM with example questions for each category and then use an appropriate prompting technique to generate question templates for each category of Socratic questioning.

We designed a series of meticulously crafted prompt templates specifically tailored to assess various types of Socratic questions generated. These templates were continuously refined to effectively facilitate the detection of code authorship, helping to mitigate the limitations of LLMs. When used alongside expert knowledge, LLMs can aid in developing complex enterprise applications through the use of knowledge generation prompting techniques, significantly enhancing the software development process [36]. Knowledge Generation is a prompting technique that boosts the ability of LLMs to generate new, relevant knowledge for the task at hand. This newly generated knowledge is then incorporated into crafting new prompts that are integrated with the task description [37]. For understanding code, this method pinpoints syntactic elements and semantic concepts, forming the foundation for developing question templates.

To facilitate effective Socratic questioning, we have developed a series of templates enhanced by various prompting techniques. These techniques allow for an exploration of different aspects of the coding process through targeted questions, aiming to probe the understanding of the code and the processes that led to its creation: (1) **Chain-of-Thought** – enables complex reasoning capabilities through intermediate reasoning steps [38], (2) **Scenario-based Simulation** – simulates real-world scenarios to test the application of code in various contexts [39], (3) **Counterfactual Thinking** – explores hypothetical alterations to understand how different conditions might affect outcomes [40], (4) **Directional Stimulus Prompting** – focuses attention on specific aspects of a problem to extract deeper insights [41], (5) **Prompt Chaining** – builds a sequence of questions that gradually increase in complexity or detail, encouraging comprehensive analysis [42], (6) **Self-Reflection** – encourages individuals to critically assess their own understanding and approach to the coding task, with the results provided as context for an LLM in subsequent prompts [43]. Here are some examples to demonstrate how these techniques are applied across different categories of Socratic questioning:

- **Conceptual Clarification** uses Scenario-based Simulation to delve into a student's understanding of

their coding process. For example, a student could be asked to explain modifications needed to handle new requirements in their code, discussing the purpose of each variable and the impact of these changes on the overall code structure and functionality.

- **Probing Assumptions** utilizes Counterfactual Thinking to explore how solutions might shift under different conditions. A typical question might involve how the student would alter their algorithm if the input constraints were changed.
- **Reasons and Evidence** applies Chain-of-Thought to uncover the logic behind specific programming choices, such as opting for a loop over a recursive solution, to delve into the student's thought process.
- **Perspective and Viewpoint** questions**,** employing techniques like Chain of Thought and Directional Stimulus Prompting, prompt students to analyse coding problems from different angles. For instance, students might be asked how their solution strategy would change if they used a stack instead of a queue, exploring their understanding of these data structures and their effects on the algorithm's behaviour.
- **Implications and Consequences** questions, using Chain of Thought and Prompt Chaining, encourage students to consider the effects of code modifications. For instance, students could explore the potential impacts of converting a recursive function to an iterative format.

Reflective Questions engage students in Self-Explanation and Reflective Analysis, asking them to recount moments of uncertainty while coding and how they addressed such challenges, promoting a deeper insight into their problem-solving process.

C. Assessment of code understanding

To evaluate the quality of student responses regarding their understanding of the code development process, the following criteria are applied:

- **Consistency and Terminology:** Evaluates whether explanations consistently use terminology throughout and provide confident, coherent explanations.
- **Technical Accuracy:** Assesses if responses are technically correct, demonstrating a clear and accurate understanding of the code's functionality.
- **Reaction to Hypothetical Modifications:** Measures the accuracy of predictions regarding the consequences of code modifications, indicating a deep understanding of code structure and functionality.
- **Development Process Insight:** Reviews detailed accounts of the development process, including the tools used, resources referenced, and debugging steps undertaken.
- **Depth of Understanding:** Determines the ability to engage with complex problem-solving related to the code and insight into advanced topics associated with the task.

Each criterion is designed to assess a student's understanding of the code development process, specifically targeting their familiarity with the code they submitted. This approach aims to determine whether the student is truly the author of the code by examining their insights into the creation and structure of the program, rather than their general programming knowledge. This focused evaluation helps identify how deeply students comprehend the workings and rationale behind their code, as well as areas where they may need further instruction or clarification. The weight assigned to each criterion in the evaluation is flexibly determined by the instructor, tailored to the specific requirements of the assignment. The results of the assessment dictate subsequent actions: if there is a high probability that the student is not the original author of the code, they are referred to another agent who assists in rewriting it. Conversely, if the assessment confirms that the student did author the code, they will receive recommendations for improvement. All interactions are recorded and the details are forwarded to the instructor for further review and action.

IV. EVALUATION

In the upcoming semester, AVERT will undergo a trial phase with a small group of students to assess its impact on both students and instructors. The evaluation process will involve the development of a comprehensive questionnaire designed to determine whether AVERT meets the criteria for a robust assessment of authorship. This questionnaire will collect feedback on various aspects of the tool, including its effectiveness in identifying the authorship of programming assignments, its usability, and its influence on students' learning experiences. Additionally, the evaluation will consider instructors' perspectives on the integration of AVERT into their teaching workflows, its efficiency in streamlining the grading process, and its overall impact on academic integrity.

To ensure a thorough evaluation, the questionnaire will focus on several key areas, including the following aspects related to AVERT's effectiveness, usability, and impact on both students and instructors:

- **Accuracy in Detecting Plagiarism:** Measuring AVERT's effectiveness in identifying instances of code plagiarism, including detecting both direct copying and more sophisticated obfuscation techniques.
- **Reduction of False Positives:** Investigating the extent to which AVERT reduces false accusations of plagiarism by distinguishing between legitimate similarities (e.g., common algorithms or libraries) and actual cases of code copying.
- **Impact on Academic Integrity:** Evaluating whether AVERT's implementation has led to a decrease in plagiarism incidents and if its presence alone serves as a deterrent for students considering dishonest practices.
- **Ease of Interaction with AVERT:** Evaluating how easy it is for students to interact with the system, particularly when answering follow-up questions or responding to inquiries about their code during the verification process.
- **Ease of Use for Instructors:** Assessing how intuitive and user-friendly AVERT is for instructors when setting up, running, and reviewing plagiarism detection reports.

- **Student Engagement:** Assessing how effectively AVERT encourages active participation and intellectual engagement with the material, particularly in coding assignments.
- **Learning Enhancement:** Evaluating if students perceive AVERT as a tool that enhances their understanding of coding principles, even though its primary focus is plagiarism detection, by prompting deeper reflection on their work.
- **Instructor Confidence in Plagiarism Detection:** Measuring the level of trust instructors place in AVERT's ability to detect plagiarism accurately and whether it enhances their confidence in identifying dishonest practices.
- **Ease of Interpretation of Plagiarism Reports:** Evaluating how easily instructors can interpret AVERT's plagiarism reports and whether the reports provide actionable insights for grading or addressing academic integrity concerns.

The findings from this initial testing phase will provide valuable insights into the potential adjustments needed to enhance AVERT's functionality and effectiveness in real-world educational settings.

V. CONCLUSION AND FUTURE WORK

Academic dishonesty remains a persistent problem in universities as students keep coming up with new ways of cheating that are untraceable. By doing so, an unending race has emerged between learners and instructors, requiring the latter to come up with detection algorithms that are increasingly sophisticated. This conflict was intensified by the advent of the generative AI, accentuating the urgent need for an approach that addresses the root causes of dishonest behaviour. A comprehensive meta-analysis that reviewed 79 studies on the reasons why students cheat has revealed that nearly all significant factors related to academic dishonesty can be attributed to issues of motivation and engagement [44]. Therefore, preventive measures should concentrate on student engagement and motivation. In this respect, AVERT, which uses generative AI to detect cheating by assessing students' understanding of their homework represents a milestone in the preservation of academic integrity. Instead of just reacting to cheating, it encourages authentic learning, by addressing the main causes of dishonest behaviour for the benefit of both students and instructors. This approach reduces the need for surveillance and punitive actions by promoting a constructive learning experience. AVERT transforms how authorship is confirmed in programming assignments, demonstrating that plagiarism prevention is possible through personalized student support based on their individual abilities.

This project is a work in progress and introduces AVERT as a component within a proactive LLM-based chatbot tutoring system, representing just the initial phase of a broader multi-agent framework. Utilizing the Langchain framework, it enhances the chatbot's capacity to reason through problems and decompose them into manageable sub-tasks. The goal of the chatbot that AVERT is part of is to assist students in solving the programming assignments without directly providing the code. The work presented in this paper demonstrates the feasibility of using a specialized agent within this framework to verify if the students have written the code themselves by verifying the students' understanding of the code development process. While this paper presents only a proof-of-concept prototype of the implementation of a novel approach for establishing authorship of code in programming assignments, future work, based on feedback from students and instructors, will focus on improving and expanding the system. Further, we will consider extending its application to advanced programming assignments and its integration in learning management systems to increase the system effectiveness and adaptability to diverse educational demands. Future developments will focus on enhancing the system to address any shortcomings and customize it to the specific needs of instructors and students.

REFERENCES

[1] J. Walker, "Measuring plagiarism: researching what students do, not what they say they do," Studies in Higher Education, vol. 35, no. 1. Informa UK Limited, pp. 41–59, Nov. 17, 2009. doi: 10.1080/03075070902912994.

[2]] D. Ison, "Academic Misconduct and the Internet," Scholarly Ethics and Publishing. IGI Global, pp. 22–51, 2019. doi: 10.4018/978-1-5225-8057-7.ch002.

[3] N. Brunelle and J. R. Hott, "Ask Me Anything," Proceedings of the 51st ACM Technical Symposium on Computer Science Education. ACM, Feb. 26, 2020. doi: 10.1145/3328778.3372658.

[4] T. Marques, N. Reis, and J. Gomes, "A Bibliometric Study on Academic Dishonesty Research," Journal of Academic Ethics, vol. 17, no. 2. Springer Science and Business Media LLC, pp. 169–191, Apr. 12, 2019. doi: 10.1007/s10805-019-09328-2.

[5] Y. K. Dwivedi et al., "Opinion Paper: 'So what if ChatGPT wrote it?' Multidisciplinary perspectives on opportunities, challenges and implications of generative conversational AI for research, practice and policy," International Journal of Information Management, vol. 71. Elsevier BV, p. 102642, Aug. 2023. doi: 10.1016/j.ijinfomgt.2023.102642.

[6] M. Alshater, "Exploring the Role of Artificial Intelligence in Enhancing Academic Performance: A Case Study of ChatGPT," SSRN Electronic Journal. Elsevier BV, 2022. doi: 10.2139/ssrn.4312358.

[7] E. Kasneci et al., "ChatGPT for good? On opportunities and challenges of large language models for education," Learning and Individual Differences, vol. 103. Elsevier BV, p. 102274, Apr. 2023. doi: 10.1016/j.lindif.2023.102274.

[8] D. R. E. Cotton, P. A. Cotton, and J. R. Shipway, "Chatting and cheating: Ensuring academic integrity in the era of ChatGPT," Innovations in Education and Teaching International, vol. 61, no. 2. Informa UK Limited, pp. 228–239, Mar. 13, 2023. doi: 10.1080/14703297.2023.2190148.

[9] S. W. Turner and S. Uludag, "Student perceptions of cheating in online and traditional classes," 2013 IEEE Frontiers in Education Conference (FIE). IEEE, Oct. 2013. doi: 10.1109/fie.2013.6685007.

[10] J. Manyrath, K. Kirubel, and T. Cruz, "Copy-Past Culture: Examining the Causes and Solutions to Source Code Plagiarism," London Journal of Social Sciences, no. 6. UKEY Consulting and Publishing Ltd, pp. 49–55, Sep. 17, 2023. doi: 10.31039/ljss.2023.6.104.

[11] J. Berrezueta-Guzman, M. Paulsen, and S. Krusche, "Plagiarism Detection and its Effect on the Learning Outcomes," 2023 IEEE 35th International Conference on Software Engineering Education and Training (CSEE&T). IEEE, Aug. 2023. doi: 10.1109/cseet58097.2023.00021.

[12] Ali, Asim M. El Tahir et al. "Overview and Comparison of Plagiarism Detection Tools." Databases, Texts, Specifications, Objects (2011). Available: https://ceur-ws.org/Vol-706/poster22.pdf.

[13]] G. Lee, J. Kim, M. Choi, R.-Y. Jang, and R. Lee, "Review of Code Similarity and Plagiarism Detection Research Studies," Applied Sciences, vol. 13, no. 20. MDPI AG, p. 11358, Oct. 16, 2023. doi: 10.3390/app132011358.

[14] A. A. Pandit and G. Toksha, "Review of Plagiarism Detection Technique in Source Code," International Conference on Intelligent Computing and Smart Communication 2019. Springer Singapore, pp. 393–405, Dec. 20, 2019. doi: 10.1007/978-981-15-0633-8_38.

[15] M. Agrawal and D. K. Sharma, "A state of art on source code plagiarism detection," 2016 2nd International Conference on Next

Generation Computing Technologies (NGCT). IEEE, Oct. 2016. doi: 10.1109/ngct.2016.7877421.

[16] R. C. Aniceto, M. Holanda, C. Castanho, and D. Da Silva, "Source Code Plagiarism Detection in an Educational Context: A Literature Mapping," 2021 IEEE Frontiers in Education Conference (FIE). IEEE, Oct. 13, 2021. doi: 10.1109/fie49875.2021.9637155.

[17] M. Horváth and E. Pietriková, "An Experimental Comparison of Three Code Similarity Tools on Over 1,000 Student Projects," 2024 IEEE 22nd World Symposium on Applied Machine Intelligence and Informatics (SAMI). IEEE, Jan. 25, 2024. doi: 10.1109/sami60510.2024.10432863.

[18] C. Ragkhitwetsagul, J. Krinke, and D. Clark, "A comparison of code similarity analysers," Empirical Software Engineering, vol. 23, no. 4. Springer Science and Business Media LLC, pp. 2464–2519, Oct. 25, 2017. doi: 10.1007/s10664-017-9564-7.

[19] M. Zakeri-Nasrabadi, S. Parsa, M. Ramezani, C. Roy, and M. Ekhtiarzadeh, "A systematic literature review on source code similarity measurement and clone detection: Techniques, applications, and challenges," Journal of Systems and Software, vol. 204. Elsevier BV, p. 111796, Oct. 2023. doi: 10.1016/j.jss.2023.111796..

[20] R. Maertens et al., "Dolos: Language‐agnostic plagiarism detection in source code," Journal of Computer Assisted Learning, vol. 38, no. 4. Wiley, pp. 1046−1061, Mar. 09, 2022. doi: 10.1111/jcal.12662.

[21] B. Denzler, F. Vahid, and A. Pang, "Style Anomalies Can Suggest Cheating in CS1 Programs," Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 2. ACM, Mar. 14, 2024. doi: 10.1145/3626253.3635519.

[22] F. Vahid, A. Pang, and B. Denzler, "Towards Comprehensive Metrics for Programming Cheat Detection," Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1. ACM, Mar. 07, 2024. doi: 10.1145/3626252.3630951.

[23] N. Tahaei and D. C. Noelle, "Automated Plagiarism Detection for Computer Programming Exercises Based on Patterns of Resubmission," Proceedings of the 2018 ACM Conference on International Computing Education Research. ACM, Aug. 08, 2018. doi: 10.1145/3230977.3231006.

[24] M. Oedingen, R. C. Engelhardt, R. Denz, M. Hammer, and W. Konen, "ChatGPT Code Detection: Techniques for Uncovering the Source of Code," AI, vol. 5, no. 3. MDPI AG, pp. 1066–1094, Jul. 02, 2024. doi: 10.3390/ai5030053.

[25] S. Biderman and E. Raff, "Fooling MOSS Detection with Pretrained Language Models," Proceedings of the 31st ACM International Conference on Information & Knowledge Management. ACM, Oct. 17, 2022. doi: 10.1145/3511808.3557079.

[26] M. Khalil and E. Er, "Will ChatGPT Get You Caught? Rethinking of Plagiarism Detection," Lecture Notes in Computer Science. Springer Nature Switzerland, pp. 475–487, 2023. doi: 10.1007/978-3-031-34411-4_32.

[27] F. Gerhardus Hattingh, A. A. K. Buitendag, and J. S. Van Der Walt, "Presenting an Alternative Source Code Plagiarism Detection Framework for Improving the Teaching and Learning of Programming," Journal of Information Technology Education: Innovations in Practice, vol. 12. Informing Science Institute, pp. 045–058, 2013. doi: 10.28945/1769.

[28] S. Mallik and A. Gangopadhyay, "Proactive and reactive engagement of artificial intelligence methods for education: a review," Frontiers in Artificial Intelligence, vol. 6. Frontiers Media SA, May 05, 2023. doi: 10.3389/frai.2023.1151391,

[29] V. Alto, Building LLM Powered Applications: Create intelligent apps and agents with large language models, Packt Publishing, 2024.

[30] S. Stapleton et al., "A Human Study of Comprehension and Code Summarization," Proceedings of the 28th International Conference on Program Comprehension. ACM, Jul. 13, 2020. doi: 10.1145/3387904.3389258.

[31] S. Stapleton et al., "A Human Study of Comprehension and Code Summarization," Proceedings of the 28th International Conference on Program Comprehension. ACM, Jul. 13, 2020. doi: 10.1145/3387904.3389258.

[32] E. Y. Chang, "Prompting Large Language Models With the Socratic Method," arXiv, 2023, doi: 10.48550/ARXIV.2303.08769.

[33] A. Webson and E. Pavlick, "Do Prompt-Based Models Really Understand the Meaning of Their Prompts?," Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.naacl-main.167.

[34] B. Chen, Z. Zhang, N. Langrené, and S. Zhu, "Unleashing the potential of prompt engineering in Large Language Models: a comprehensive review," 2023, arXiv. doi: 10.48550/ARXIV.2310.14735.

[35] T. Wu, M. Terry, and C. J. Cai, "AI Chains: Transparent and Controllable Human-AI Interaction by Chaining Large Language Model Prompts," CHI Conference on Human Factors in Computing Systems. ACM, Apr. 29, 2022. doi: 10.1145/3491102.3517582.

[36] Martin Fowler, 13 April 2023, "An example of LLM prompting for programming", martinFowler.com. [Online]. Available: https://martinfowler.com/articles/2023-chatgpt-xu-hao.html

[37] J. Liu et al., "Generated Knowledge Prompting for Commonsense Reasoning," Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, 2022. doi: 10.18653/v1/2022.acl-long.225.

[38] J. Wei et al., "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," 2022, arXiv. doi: 10.48550/ARXIV.2201.11903.

[39] D. Harel, G. Katz, A. Marron, and S. Szekely, "On Augmenting Scenario-Based Modelling with Generative AI," 2024, arXiv. doi: 10.48550/ARXIV.2401.02245.

[40] J. Kim, Y. J. Kim, and Y. M. Ro, "What if...?: Thinking Counterfactual Keywords Helps to Mitigate Hallucination in Large Multi-modal Models," 2024, arXiv. doi: 10.48550/ARXIV.2403.13513.

[41] Z. Li, B. Peng, P. He, M. Galley, J. Gao, and X. Yan, "Guiding Large Language Models via Directional Stimulus Prompting," 2023, arXiv. doi: 10.48550/ARXIV.2302.11520.

[42] S. Sun, R. Yuan, Z. Cao, W. Li, and P. Liu, "Prompt Chaining or Stepwise Prompt? Refinement in Text Summarization," 2024, arXiv. doi: 10.48550/ARXIV.2406.00507.

[43] M. Renze and E. Guven, "Self-Reflection in LLM Agents: Effects on Problem-Solving Performance," 2024, arXiv. doi: 10.48550/ARXIV.2405.06682.

[44] M. R. Krou, C. J. Fong, and M. A. Hoff, "Achievement Motivation and Academic Dishonesty: A Meta-Analytic Investigation," Educational Psychology Review, vol. 33, no. 2. Springer Science and Business Media LLC, pp. 427–458, Aug. 01, 2020. doi: 10.1007/s10648-020-09557-7.